

### Assignment-3

Naheed Deedar

Id : 801925

-- =====

-- SQL Server " Class 3 Homework

-- BikeStores Sample Database

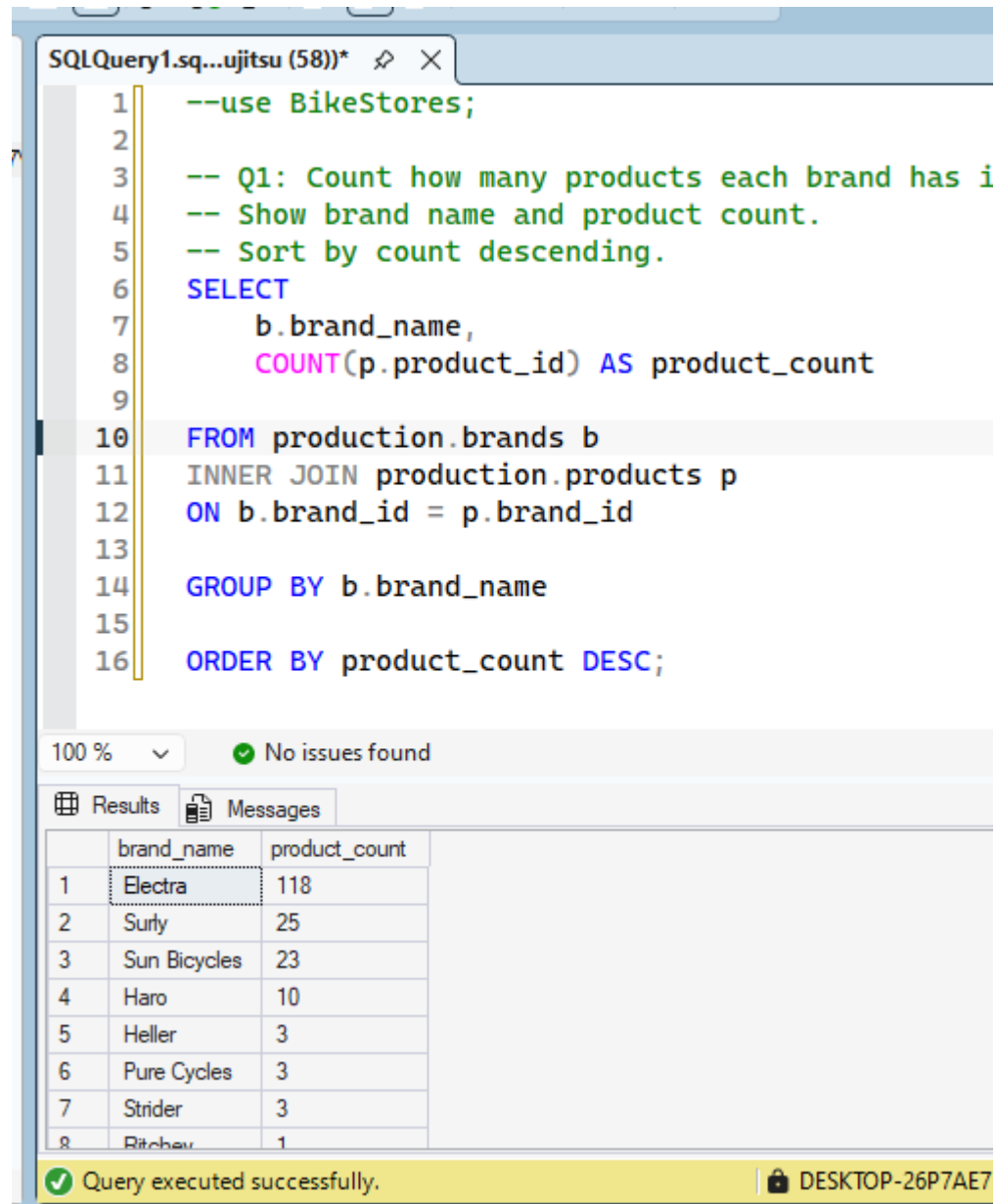
-- Topics: GROUP BY · HAVING · Subqueries · EXISTS

-- =====

-- Q1: Count how many products each brand has in the catalog.

-- Show brand name and product count.

-- Sort by count descending.



The screenshot shows a SQL IDE window titled "SQLQuery1.sql...ujitsu (58))\*". The query editor contains the following SQL code:

```
1  --use BikeStores;
2
3  -- Q1: Count how many products each brand has i
4  -- Show brand name and product count.
5  -- Sort by count descending.
6  SELECT
7      b.brand_name,
8      COUNT(p.product_id) AS product_count
9
10 FROM production.brands b
11 INNER JOIN production.products p
12 ON b.brand_id = p.brand_id
13
14 GROUP BY b.brand_name
15
16 ORDER BY product_count DESC;
```

Below the query editor, the "Results" tab is active, displaying a table with the following data:

|   | brand_name   | product_count |
|---|--------------|---------------|
| 1 | Electra      | 118           |
| 2 | Surly        | 25            |
| 3 | Sun Bicycles | 23            |
| 4 | Haro         | 10            |
| 5 | Heller       | 3             |
| 6 | Pure Cycles  | 3             |
| 7 | Strider      | 3             |
| 8 | Ritchey      | 1             |

At the bottom of the IDE, a status bar indicates "Query executed successfully." and the system name "DESKTOP-26P7AE7" is visible.

-- Q2: For each category, show:

-- category name,

-- total number of products,

-- cheapest price,

-- most expensive price,

-- average price (rounded to 2 decimals).

-- Sort by average price descending.

```
SQLQuery-1.s...fujitsu (58) ✕
2  -- Q2: For each category, show:
3  -- category name,
4  -- total number of products,
5  -- cheapest price,
6  -- most expensive price,
7  -- average price (rounded to 2 decimals).
8  -- Sort by average price descending.
9
10 SELECT
11     c.category_name,
12     COUNT(p.product_id) AS total_products,
13     MIN(p.list_price) AS cheapest_price,
14     MAX(p.list_price) AS most_expensive_price,
15     ROUND(AVG(p.list_price), 2) AS average_price
16 FROM production.categories c
17
18 INNER JOIN production.products p
19 ON c.category_id = p.category_id
20
21 GROUP BY c.category_name
22
23 ORDER BY average_price DESC;
```

100 %

No issues found



Results



Messages

|   | category_name       | total_products | cheapest_price | most_expensive_price | average_price |
|---|---------------------|----------------|----------------|----------------------|---------------|
| 1 | Electric Bikes      | 6              | 1559.99        | 2999.99              | 2593.320000   |
| 2 | Cyclocross Bicycles | 4              | 1549.00        | 1680.99              | 1582.000000   |
| 3 | Mountain Bikes      | 25             | 379.99         | 2599.00              | 1262.630000   |
| 4 | Road Bikes          | 7              | 749.99         | 1899.00              | 1246.000000   |
| 5 | Cruisers Bicycles   | 78             | 250.99         | 2999.99              | 730.410000    |
| 6 | Comfort Bicycles    | 30             | 416.99         | 2599.99              | 682.120000    |
| 7 | Children Bicycles   | 36             | 89.99          | 489.99               | 305.270000    |

Query executed successfully.

DESKTOP-26P7AE7 (17.0 RTM) | DE

-- Q3: Show the number of orders placed per order status.

-- Display the status value and order count.

-- Sort by order\_status ascending.

The screenshot shows a SQL query editor window titled "SQLQuery-2.s...fujitsu (58)". The query is as follows:

```
1
2  -- Q3: Show the number of orders placed per order status.
3  -- Display the status value and order count.
4  -- Sort by order_status ascending
5  SELECT
6      order_status,
7      COUNT(order_id) AS order_count
8  FROM sales.orders
9
10 GROUP BY order_status
11
12 ORDER BY order_status ASC;
```

Below the query editor, the "Results" tab is active, displaying a table with the following data:

|   | order_status | order_count |
|---|--------------|-------------|
| 1 | 1            | 62          |
| 2 | 2            | 63          |
| 3 | 3            | 45          |
| 4 | 4            | 1445        |

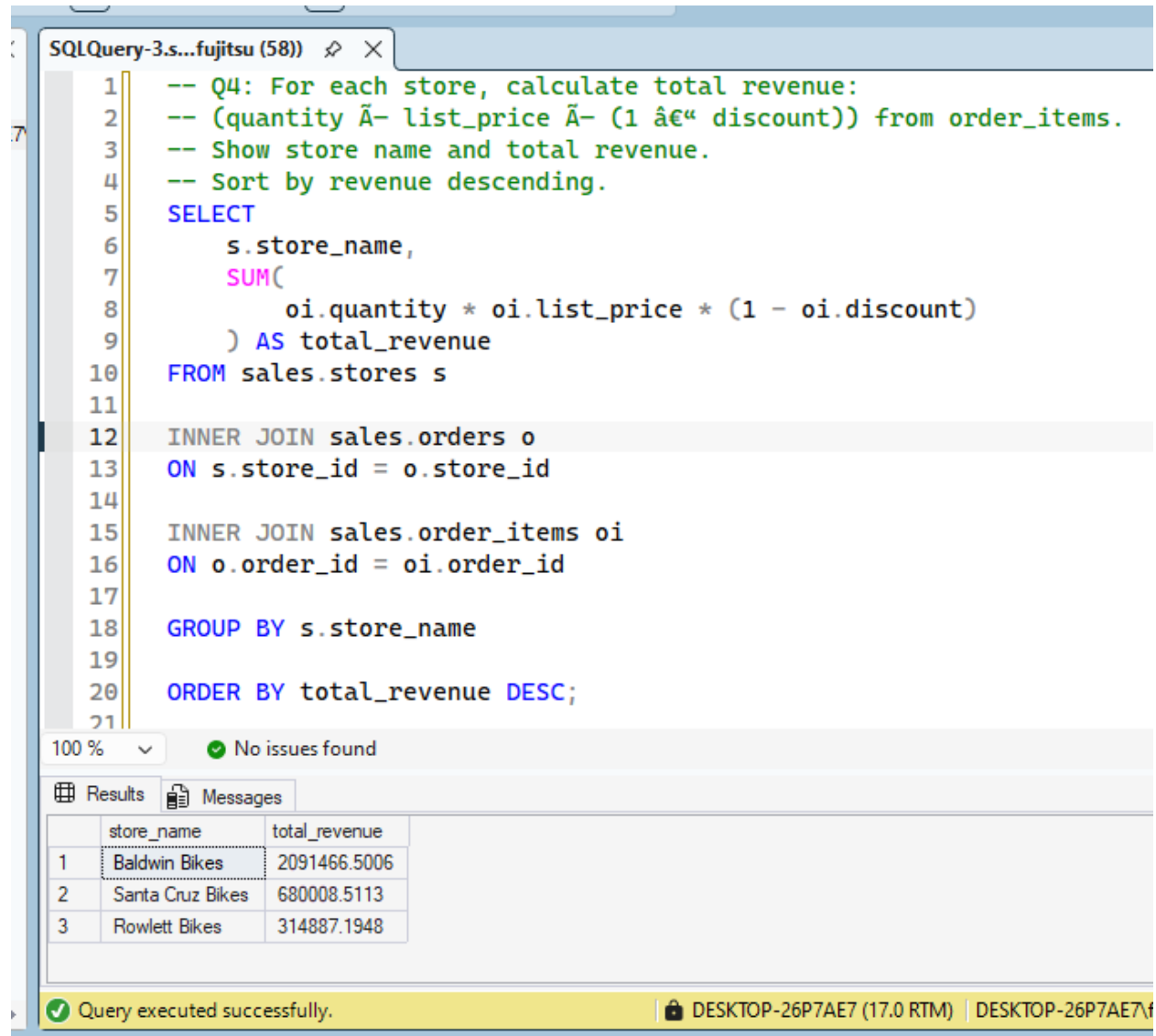
At the bottom of the window, a status bar indicates "Query executed successfully." and the system information "DESKTOP-26P7AE7 (17.0 RTM) | DESK".

- Q4: For each store, calculate total revenue:

-- (quantity \* list\_price \* (1 - discount)) from order\_items.

-- Show store name and total revenue.

-- Sort by revenue descending.



The screenshot shows a SQL query editor window titled "SQLQuery-3.s...fujitsu (58)". The query is as follows:

```
1  -- Q4: For each store, calculate total revenue:
2  -- (quantity * list_price * (1 - discount)) from order_items.
3  -- Show store name and total revenue.
4  -- Sort by revenue descending.
5  SELECT
6      s.store_name,
7      SUM(
8          oi.quantity * oi.list_price * (1 - oi.discount)
9      ) AS total_revenue
10 FROM sales.stores s
11
12 INNER JOIN sales.orders o
13 ON s.store_id = o.store_id
14
15 INNER JOIN sales.order_items oi
16 ON o.order_id = oi.order_id
17
18 GROUP BY s.store_name
19
20 ORDER BY total_revenue DESC;
21
```

Below the query editor, the "Results" tab is active, displaying a table with the following data:

|   | store_name       | total_revenue |
|---|------------------|---------------|
| 1 | Baldwin Bikes    | 2091466.5006  |
| 2 | Santa Cruz Bikes | 680008.5113   |
| 3 | Rowlett Bikes    | 314887.1948   |

At the bottom of the window, a status bar indicates "Query executed successfully." and the system information "DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\..." is visible.

- Q5: Show total number of products per brand per model year.
- Display brand name, model year, and product count.
- Sort by brand name then model year.

SQLQuery-4.s...fujitsu (58) ✕

```
2  -- Q5: Show total number of products per brand per model year.
3  -- Display brand name, model year, and product count.
4  -- Sort by brand name then model year.
5  SELECT
6      b.brand_name,
7      p.model_year,
8      COUNT(p.product_id) AS product_count
9  FROM production.brands b
10
11  INNER JOIN production.products p
12  ON b.brand_id = p.brand_id
13
14  GROUP BY
15      b.brand_name,
16      p.model_year
17
18  ORDER BY
19      b.brand_name ASC,
20      p.model_year ASC;
```

100 % ▾ ✓ No issues found

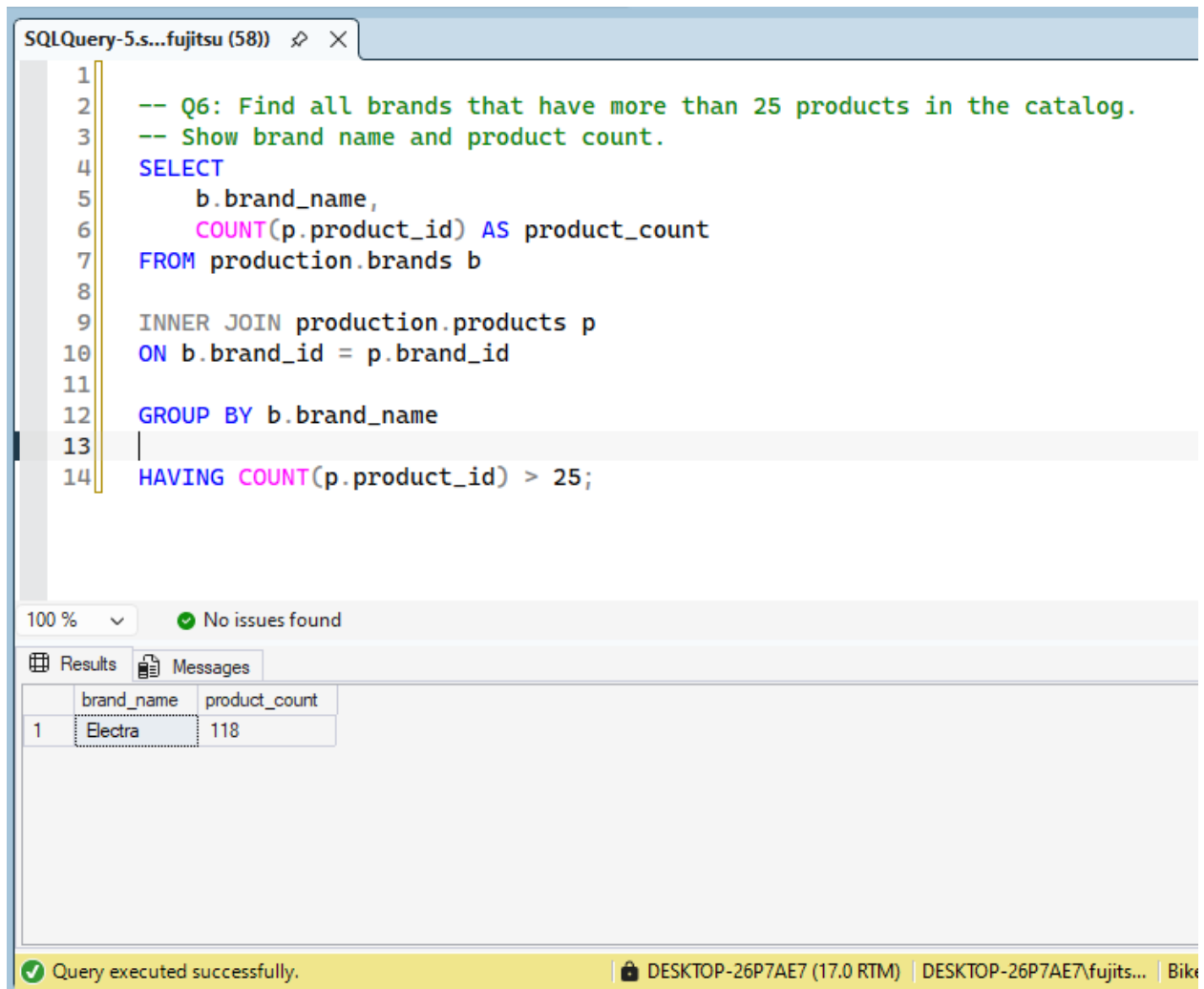
Results Messages

|   | brand_name | model_year | product_count |
|---|------------|------------|---------------|
| 1 | Electra    | 2016       | 12            |
| 2 | Electra    | 2017       | 16            |
| 3 | Electra    | 2018       | 90            |
| 4 | Haro       | 2017       | 10            |

✓ Query executed successfully. | DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\fujits... | Biki

-- Q6: Find all brands that have more than 25 products in the catalog.

-- Show brand name and product count.



The screenshot shows a SQL query editor window titled "SQLQuery-5.s...fujitsu (58)". The query is as follows:

```
1
2  -- Q6: Find all brands that have more than 25 products in the catalog.
3  -- Show brand name and product count.
4  SELECT
5      b.brand_name,
6      COUNT(p.product_id) AS product_count
7  FROM production.brands b
8
9  INNER JOIN production.products p
10 ON b.brand_id = p.brand_id
11
12 GROUP BY b.brand_name
13
14 HAVING COUNT(p.product_id) > 25;
```

Below the query editor, there is a status bar showing "100 %" and "No issues found". Below that, there are tabs for "Results" and "Messages". The "Results" tab is active, showing a table with the following data:

|   | brand_name | product_count |
|---|------------|---------------|
| 1 | Electra    | 118           |

At the bottom of the window, a yellow status bar indicates "Query executed successfully." followed by system information: "DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\fujitsu... | Bike".

-- Q7: Among products from year 2018 only,  
-- find categories whose average price is above \$1500.  
-- Show category name, product count, and average price.

The screenshot shows a SQL Query Editor window titled "SQLQuery-6.s...fujitsu (58)". The query is as follows:

```
1  -- Q7: Among products from year 2018 only,  
2  -- find categories whose average price is above $1500.  
3  -- Show category name, product count, and average price.  
4  SELECT  
5      c.category_name,  
6      COUNT(p.product_id) AS product_count,  
7      ROUND(AVG(p.list_price), 2) AS average_price  
8  FROM production.categories c  
9  
10 INNER JOIN production.products p  
11 ON c.category_id = p.category_id  
12  
13 WHERE p.model_year = 2018  
14  
15 GROUP BY c.category_name  
16  
17 HAVING AVG(p.list_price) > 1500  
18  
19 ORDER BY average_price DESC;
```

Below the query editor, the "Results" tab is active, displaying a table with the following data:

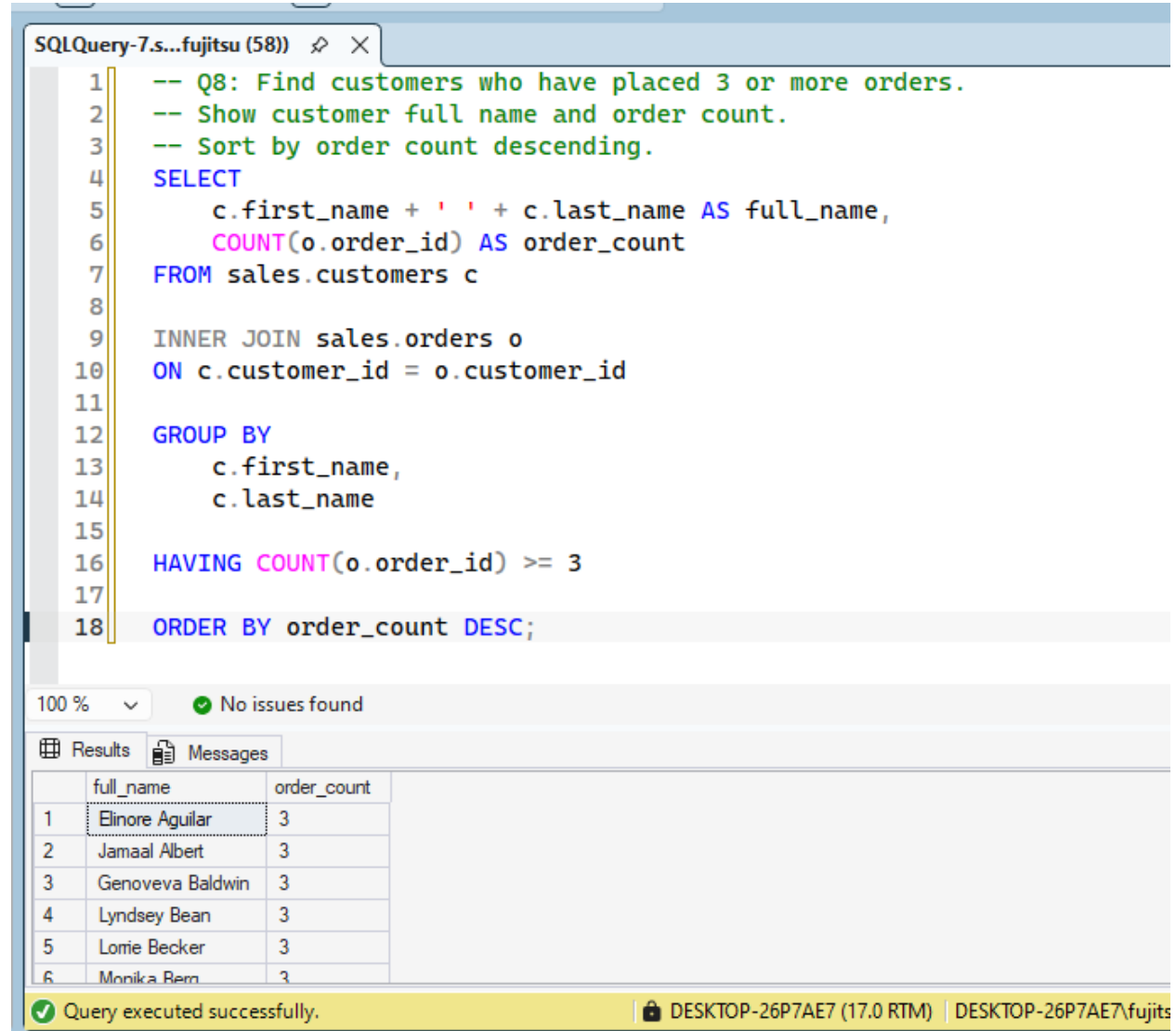
|   | category_name       | product_count | average_price |
|---|---------------------|---------------|---------------|
| 1 | Electric Bikes      | 5             | 2799.990000   |
| 2 | Mountain Bikes      | 8             | 1817.000000   |
| 3 | Cyclocross Bicycles | 2             | 1549.000000   |

The status bar at the bottom indicates "Query executed successfully." and shows the system path "DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\fujitsu... | BikeSto".

-- Q8: Find customers who have placed 3 or more orders.

-- Show customer full name and order count.

-- Sort by order count descending.



The screenshot shows the SQL Developer interface. The top pane displays a SQL query for finding customers with 3 or more orders. The bottom pane shows the results of the query, which are sorted by order count in descending order. The results table has two columns: full\_name and order\_count. The first six rows of results are visible.

```
SQLQuery-7.s...fujitsu (58) ✕
1  -- Q8: Find customers who have placed 3 or more orders.
2  -- Show customer full name and order count.
3  -- Sort by order count descending.
4  SELECT
5      c.first_name + ' ' + c.last_name AS full_name,
6      COUNT(o.order_id) AS order_count
7  FROM sales.customers c
8
9  INNER JOIN sales.orders o
10 ON c.customer_id = o.customer_id
11
12 GROUP BY
13     c.first_name,
14     c.last_name
15
16 HAVING COUNT(o.order_id) >= 3
17
18 ORDER BY order_count DESC;
```

100 % ▾ No issues found

Results Messages

|   | full_name        | order_count |
|---|------------------|-------------|
| 1 | Elinore Aguilar  | 3           |
| 2 | Jamaal Albert    | 3           |
| 3 | Genoveva Baldwin | 3           |
| 4 | Lyndsey Bean     | 3           |
| 5 | Lomie Becker     | 3           |
| 6 | Monika Bern      | 3           |

✓ Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\fujitsu

-- Q9: Find all products whose list price is higher than  
-- the average list price of all products.  
-- Show product name and price.  
-- Sort by price descending.

SQLQuery-7.s...fujitsu (58) ✕

```
1  -- Q9: Find all products whose list price is higher than
2  -- the average list price of all products.
3  -- Show product name and price.
4  -- Sort by price descending.
5  SELECT
6      product_name,
7      list_price
8  FROM production.products
9  WHERE list_price > (
10     SELECT AVG(list_price)
11     FROM production.products
12 )
13 ORDER BY list_price DESC;
```

100 % ▾ No issues found

Results Messages

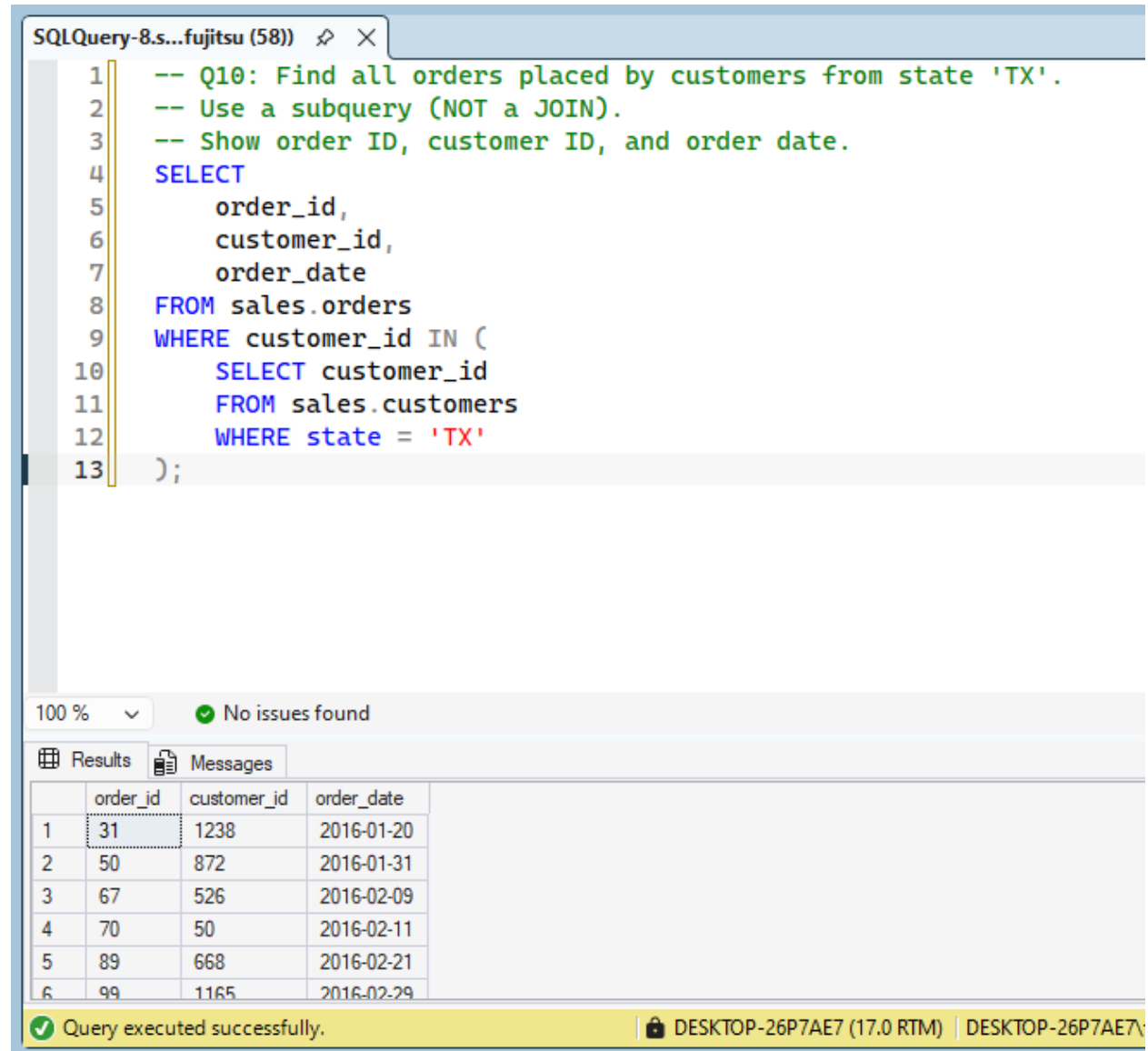
|   | product_name                              | list_price |
|---|-------------------------------------------|------------|
| 1 | Electra Townie Commute Go! - 2018         | 2999.99    |
| 2 | Electra Townie Commute Go! Ladies' - 2018 | 2999.99    |
| 3 | Electra Townie Commute Go! - 2018         | 2999.99    |
| 4 | Electra Townie Commute Go! Ladies' - 2018 | 2999.99    |
| 5 | Electra Loft Go! 8i - 2018                | 2799.99    |
| 6 | Electra Townie Go! 8i - 2017/2018         | 2599.99    |

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7A

-- Q10: Find all orders placed by customers from state 'TX'.

-- Use a subquery (NOT a JOIN).

-- Show order ID, customer ID, and order date.



The screenshot shows the SQL Developer interface. The top pane displays a SQL query for Q10, which uses a subquery to find orders from customers in Texas. The bottom pane shows the results of the query, a table with 6 rows and 4 columns: order\_id, customer\_id, and order\_date. A status bar at the bottom indicates the query was executed successfully.

```
1  -- Q10: Find all orders placed by customers from state 'TX'.
2  -- Use a subquery (NOT a JOIN).
3  -- Show order ID, customer ID, and order date.
4  SELECT
5      order_id,
6      customer_id,
7      order_date
8  FROM sales.orders
9  WHERE customer_id IN (
10     SELECT customer_id
11     FROM sales.customers
12     WHERE state = 'TX'
13 );
```

100 % No issues found

Results Messages

|   | order_id | customer_id | order_date |
|---|----------|-------------|------------|
| 1 | 31       | 1238        | 2016-01-20 |
| 2 | 50       | 872         | 2016-01-31 |
| 3 | 67       | 526         | 2016-02-09 |
| 4 | 70       | 50          | 2016-02-11 |
| 5 | 89       | 668         | 2016-02-21 |
| 6 | 99       | 1165        | 2016-02-29 |

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\

-- Q11: For each brand, show its average price,  
-- but only for brands whose average price exceeds overall product average.  
-- Use a subquery in FROM (derived table).  
-- Show brand name and average price.

The screenshot shows a SQL query editor window titled "SQLQuery-10....fujitsu (58)". The query is as follows:

```
1  -- Q11: For each brand, show its average price,  
2  -- but only for brands whose average price exceeds overall product average.  
3  -- Use a subquery in FROM (derived table).  
4  -- Show brand name and average price.  
5  SELECT  
6      b.brand_name,  
7      avg_table.avg_price  
8  FROM (  
9      SELECT  
10         p.brand_id,  
11         AVG(p.list_price) AS avg_price  
12     FROM production.products p  
13     GROUP BY p.brand_id  
14 ) AS avg_table  
15  
16 INNER JOIN production.brands b  
17 ON avg_table.brand_id = b.brand_id  
18  
19 WHERE avg_table.avg_price > (  
20     SELECT AVG(list_price)  
21     FROM production.products  
22 )
```

Below the query editor, there is a status bar showing "100 %", "No issues found", and a "Results" tab. The "Results" tab displays the following data:

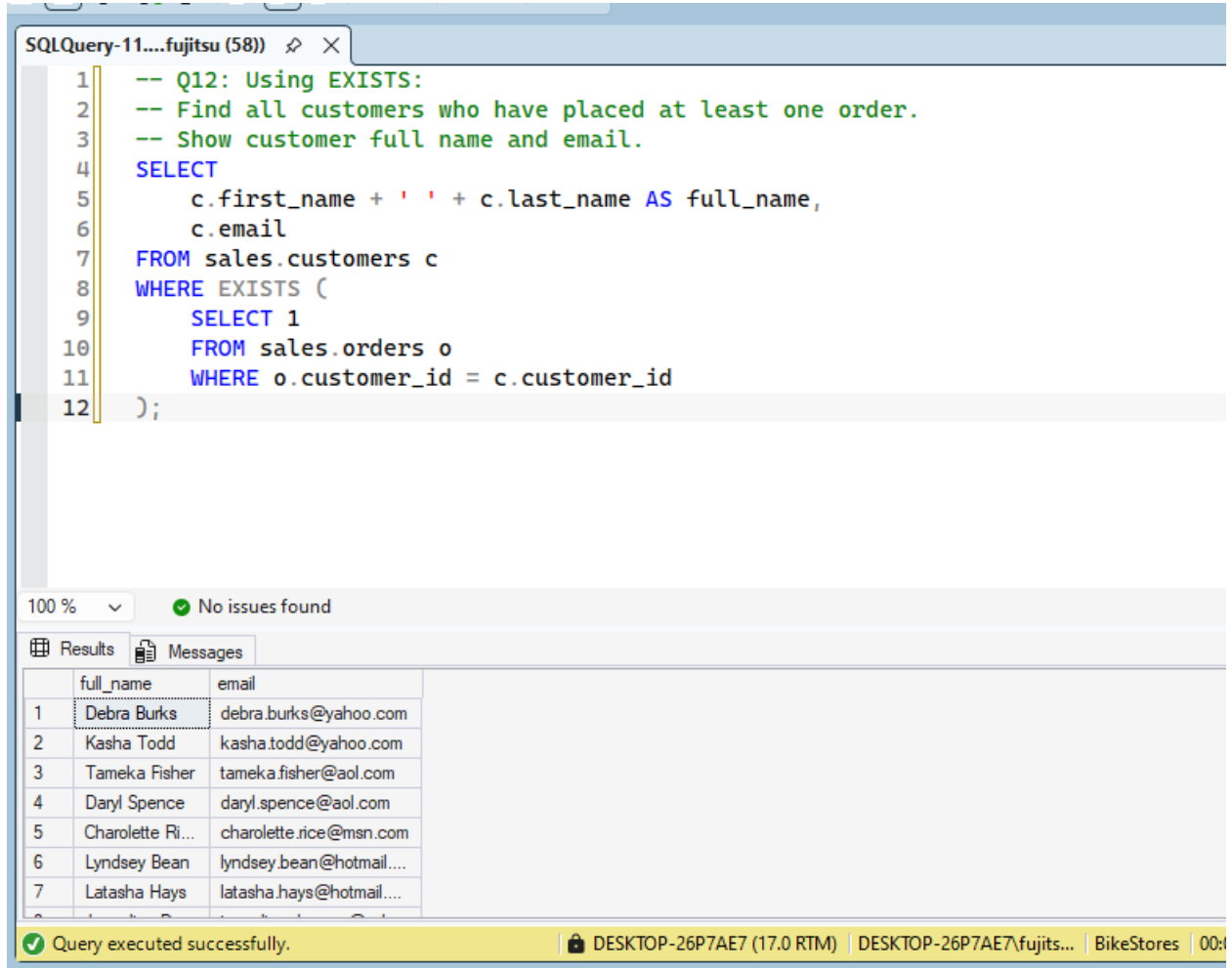
|   | brand_name | avg_price   |
|---|------------|-------------|
| 1 | Heller     | 2172.996666 |
| 2 | Surly      | 1331.753600 |

At the bottom of the window, a message bar states "Query executed successfully." and the system tray shows "DESKTOP-26P7AE7 (17.0 RTM)", "DESKTOP-26P7AE7\fujits...", "BikeStores", and "0".

-- Q12: Using EXISTS:

-- Find all customers who have placed at least one order.

-- Show customer full name and email.



The screenshot shows a SQL query editor window titled "SQLQuery-11....fujitsu (58)". The query is as follows:

```
1  -- Q12: Using EXISTS:
2  -- Find all customers who have placed at least one order.
3  -- Show customer full name and email.
4  SELECT
5      c.first_name + ' ' + c.last_name AS full_name,
6      c.email
7  FROM sales.customers c
8  WHERE EXISTS (
9      SELECT 1
10     FROM sales.orders o
11     WHERE o.customer_id = c.customer_id
12 );
```

Below the query editor, there is a status bar indicating "100 %", "No issues found", and tabs for "Results" and "Messages". The "Results" tab is active, displaying a table with the following data:

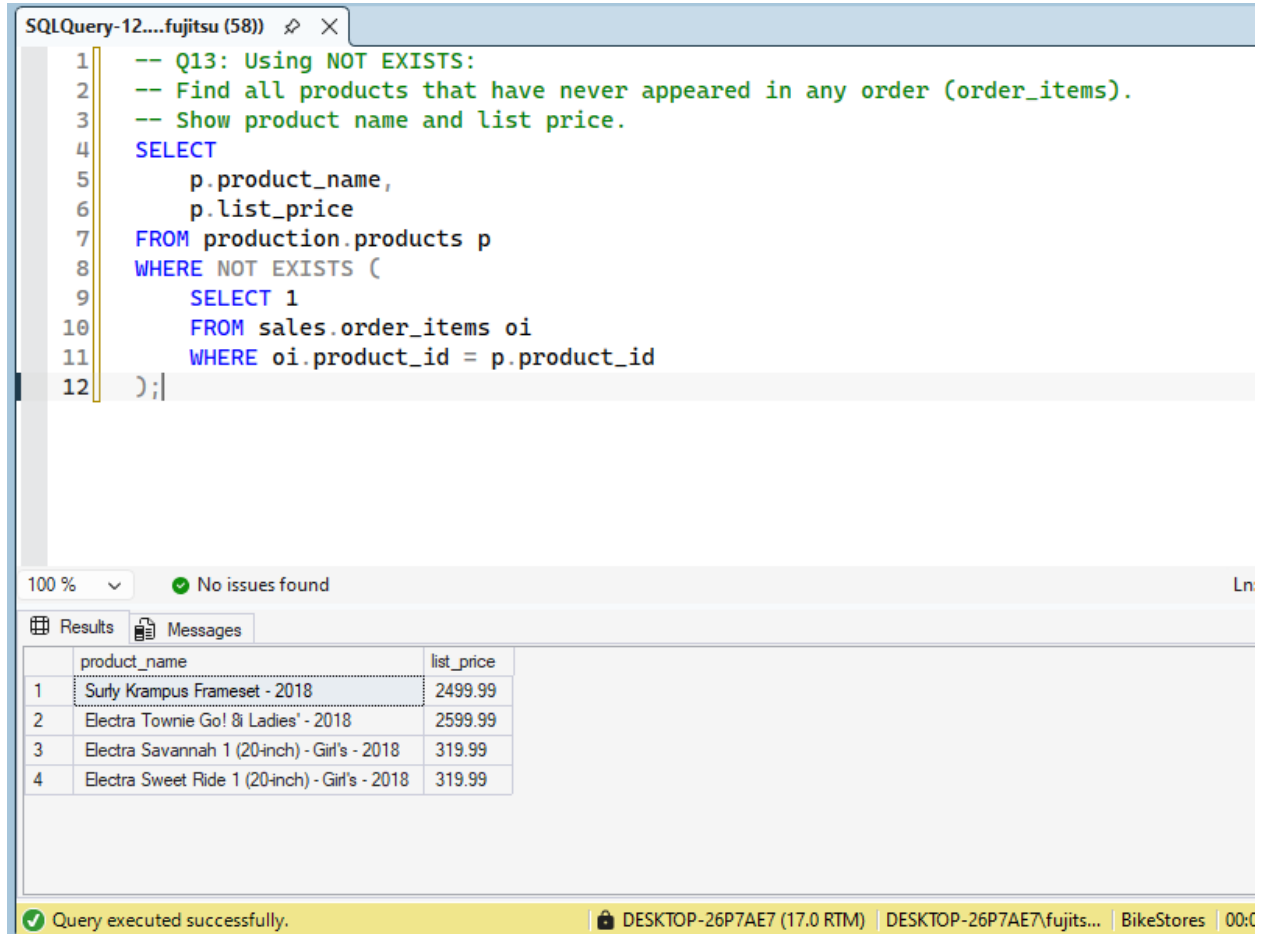
|   | full_name        | email                    |
|---|------------------|--------------------------|
| 1 | Debra Burks      | debra.burks@yahoo.com    |
| 2 | Kasha Todd       | kasha.todd@yahoo.com     |
| 3 | Tameka Fisher    | tameka.fisher@aol.com    |
| 4 | Daryl Spence     | daryl.spence@aol.com     |
| 5 | Charolette Ri... | charolette.rice@msn.com  |
| 6 | Lyndsey Bean     | lyndsey.bean@hotmail.... |
| 7 | Latasha Hays     | latasha.hays@hotmail.... |

At the bottom of the window, a yellow status bar indicates "Query executed successfully." and shows the system path "DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\fujitsu... | BikeStores | 00:".

-- Q13: Using NOT EXISTS:

-- Find all products that have never appeared in any order (order\_items).

-- Show product name and list price.



The screenshot shows the SQL Developer interface. The top pane displays a SQL query for Q13, which uses a NOT EXISTS clause to find products that have never appeared in any order. The query selects the product name and list price from the production.products table, excluding those found in the sales.order\_items table. The bottom pane shows the results of the query, which are four rows of product data. A status bar at the bottom indicates the query was executed successfully.

```
1  -- Q13: Using NOT EXISTS:
2  -- Find all products that have never appeared in any order (order_items).
3  -- Show product name and list price.
4  SELECT
5      p.product_name,
6      p.list_price
7  FROM production.products p
8  WHERE NOT EXISTS (
9      SELECT 1
10     FROM sales.order_items oi
11     WHERE oi.product_id = p.product_id
12 ) ;
```

100 %    No issues found    Ln

Results    Messages

|   | product_name                                   | list_price |
|---|------------------------------------------------|------------|
| 1 | Surly Krampus Frameset - 2018                  | 2499.99    |
| 2 | Electra Townie Go! 8i Ladies' - 2018           | 2599.99    |
| 3 | Electra Savannah 1 (20-inch) - Girl's - 2018   | 319.99     |
| 4 | Electra Sweet Ride 1 (20-inch) - Girl's - 2018 | 319.99     |

✓ Query executed successfully.    DESKTOP-26P7AE7 (17.0 RTM)    DESKTOP-26P7AE7\fujits...    BikeStores    00:0